

AWS EC2 & Nginx Deployment

Prepared By: [Abin Xavier]

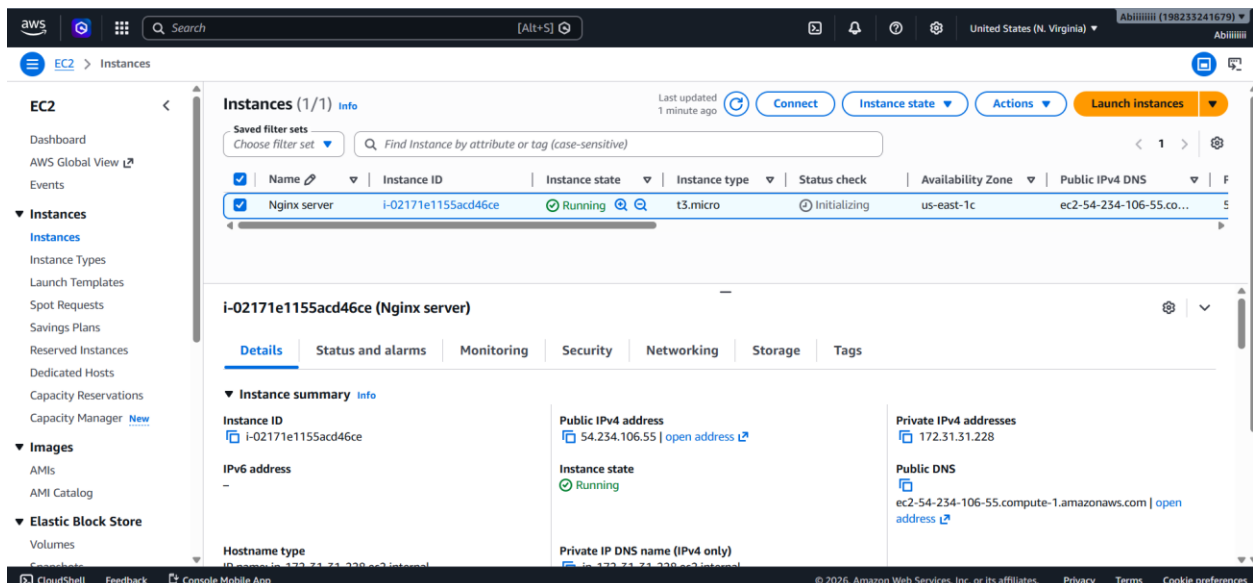
Date: June 17, 2026

Project Overview

This document outlines the successful provisioning, configuration, and deployment of a cloud-hosted web server using Amazon Web Services (AWS). A Linux-based EC2 virtual instance was launched, secured, and linked via SSH using a Windows 11 local environment. Finally, a Nginx HTTP server was compiled and verified as operational.

Step 1: AWS Free Tier Account & EC2 Dashboard Access

An AWS Free Tier account was utilized to access the management console. The setup process began by navigating directly to the **EC2** core infrastructure service.



Step 2: Provisioning the Linux EC2 Instance

Within the EC2 Launch wizard, a virtual resource named `Nginx server` was configured using the following specifications:

- **Instance Name:** Nginx server
- **Operating System (AMI):** Amazon Linux 2023 (Free tier eligible)
- **Hardware Profile (Instance Type):** t3.micro architecture
- **Key Pair Generation:** A cryptographic key pair named my_aws_key was requested and downloaded locally as a .pem file to handle Windows 11 native OpenSSH authentication.
- **Firewall Rules (Security Groups):**
 - * **Port 22 (SSH):** Enabled for remote command-line access.
 - **Port 80 (HTTP):** Enabled from anywhere (0.0.0.0/0) to allow global web requests.

Step 3: Isolating Key Pair Permissions on Windows 11

Windows 11 handles file system security tightly. To prevent an OpenSSH Unprotected Private Key File rejection error, key restrictions were explicitly applied via PowerShell using the Windows Access Control List (icacls.exe):

DOS

```
icacls.exe "my_aws_key.pem" /reset
icacls.exe "my_aws_key.pem" /grant:r "%username%: R"
icacls.exe "my_aws_key.pem" /inheritance:r
```

Step 4: Establishing Remote SSH Connectivity

Using the native Windows 11 Terminal application, a secure shell handshake was initiated targeting the host endpoint string [ec2-user@54.234.106.55](#).

Bash

```
ssh -i "my_aws_key.pem" ec2-user@54.234.106.55
```

Upon confirming host authenticity (yes), the shell environment was redirected into the isolated cloud Linux server container.

Step 6: Public Target Verification

The deployment lifecycle completed by copying the auto-allocated public IPv4 address **54.234.106.55** directly out of the AWS Instance Summary dashboard module.

The address string was fed into a local web browser interface over an unencrypted channel (<http://54.234.106.55>). The instance accurately routed the standard traffic into the functional Nginx cluster engine.

54.234.106.55



Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

Conclusion

This project provided practical knowledge of AWS EC2 provisioning, Linux server administration, SSH connectivity, security group configuration, and web server deployment using Nginx.